

Assignment 2 Report

Yusuf Emir Cömert
2220765023

Methods

All experiments in this study were conducted using the following settings:

```
learning_rates = [0.001, 0.005, 0.01]
batch_sizes = [32, 64]
num_epochs = 50
```

This results in a total of 6 experiments for each method, combining the different learning rates and batch sizes. No data augmentation was applied during training.

Input: All images are resized to **150x150** pixels with 3 color channels (RGB).

Method 1: Convolutional Neural Network (CNN)

In this method, I designed a Convolutional Neural Network (CNN) from scratch to classify the food images in the dataset. My model has **5 convolutional layers** and **2 fully connected (FC) layers**. I also used **batch normalization** and **max pooling** in my network. All experiments were performed with **50 epochs**.

Convolutional Layers: There are 5 convolutional blocks in the model. Each block has:

- A convolution layer with **3x3 kernel size** and padding 1.
- A batch normalization layer.
- A ReLU activation function.

After some blocks, I applied **max pooling** with size 2x2 to reduce the image size. For example:

- After block 2: size goes from 150 to 75.
- After block 3: from 75 to 37.
- After block 4: from 37 to 18.
- After block 5: from 18 to 9.

Dropout: I added a **dropout layer** with probability 0.5 after the convolutional blocks. This helps to prevent overfitting.

Adaptive Average Pooling: Then, I used **adaptive average pooling** to change the feature map size from 9x9 to 1x1. This makes the network smaller and faster, and also makes the algorithm to run faster.

Fully Connected Layers: After flattening, I added 2 fully connected layers:

- First FC layer: 512 units with ReLU and dropout (p=0.5).
- Second FC layer: outputs **11 classes** for classification.

Activation Function: I used **ReLU** after every layer to add non-linearity.

Loss and Optimizer: I used **CrossEntropyLoss** because this is a multi-class problem. As an optimizer, I used **Stochastic Gradient Descent (SGD)** with learning rates **0.001, 0.005, and 0.01** with **32 and 64 batch sizes** which makes **6 experiment**.

Augmentation: I have not used any augmentations in my project.

Model Code Snippet:

```
self.conv_block1 = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
)
...
self.fc = nn.Sequential(
    nn.Flatten(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)
...
```

Method 2: Residual Convolutional Neural Network (Residual CNN)

In this method, I used a Residual Convolutional Neural Network (Residual CNN) structure. This architecture includes both normal convolution layers and residual blocks. Residual blocks help the model train better by allowing the gradient to flow more easily. This is important because it avoids the vanishing gradient problem in deep networks.

The structure starts with two convolution layers:

- **Layer 1:** Conv2d(3, 32, kernel_size=3, padding=1), followed by Batch Normalization and ReLU.
- **Layer 2:** Conv2d(32, 64, kernel_size=3, padding=1), BatchNorm, ReLU, and MaxPool2d(2).

After that, I added my first residual block:

```
self.res_block1 = ResidualBlock(64, 64)
```

Next, there is another convolution layer and a second residual block:

- **Layer 3:** Conv2d(64, 128, kernel_size=3, padding=1), BatchNorm, ReLU, and MaxPool2d(2).

Then I added the second residual block:

```
self.res_block2 = ResidualBlock(128, 128)
```

To reduce the spatial size, I applied `AdaptiveAvgPool2d((1,1))`. This makes the feature map size 1x1 before sending it to the fully connected layers, and again to make it run faster.

Activation Function: ReLU is used in all layers. **Loss Function:** I used CrossEntropyLoss as shown below:

```
criterion = nn.CrossEntropyLoss()
```

For the fully connected (FC) part:

- First FC layer: `Linear(128, 512)`, followed by ReLU and Dropout (0.5).
- Second FC layer: `Linear(512, 11)` to output the 11 food classes.

Optimizer: Stochastic gradient descent optimizer was selected with different learning rates:

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9, weight_decay=1e-5)
```

The residual blocks are implemented as follows: each block has two convolution layers with BatchNorm and ReLU. There is also a skip connection that adds the input directly to the output:

```
out += identity
```

This skip connection is the main idea of residual learning. It helps the network to learn better and faster.

Model Code Snippet:

```
class ResidualCNN(nn.Module):
    def __init__(self, num_classes=11):
        super(ResidualCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU()
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.res_block1 = ResidualBlock(64, 64)

        self.layer3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.res_block2 = ResidualBlock(128, 128)

        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )
        ...
```

Method 3: Transfer Learning with MobileNetV2

In this method, I used **Transfer Learning** with the pre-trained **MobileNetV2** network from PyTorch. Transfer learning helps to improve accuracy and reduce training time by using weights that are already trained on a big dataset (ImageNet).

What is Fine-Tuning?

Fine-tuning is a transfer learning technique where we take a **pre-trained model** (usually trained on a large dataset like ImageNet) and adapt it to a new, smaller dataset related to our specific task (e.g., classifying food images from the Food-11 dataset).

Instead of training a model from scratch (which requires huge amounts of data and compute), fine-tuning **reuses the knowledge** (features) that the model has already learned from the large dataset and adapts it to the new problem.

Why Should We Fine-Tune?

- **Saves Time and Compute:** Training deep models from scratch takes days or weeks. Fine-tuning can achieve good performance in hours.
- **Better Performance on Small Datasets:** Small datasets easily cause overfitting if trained from scratch. Fine-tuning leverages the general patterns learned from large datasets, preventing overfitting.

- **Fast Convergence:** Since the model starts from a good initialization (pre-trained weights), it converges faster than random initialization.

Why Do We Freeze the Rest and Train Only the Fully Connected (FC) Layers?

The early layers of CNN models (like MobileNetV2) learn **general features** (such as edges, textures, and colors) that are useful for many tasks.

The last layers (especially the FC layers) are **task-specific** (e.g., classifying 1000 ImageNet classes). For our case (11 food classes), they are not suitable.

So, we **freeze** the general layers and only **train** the last FC layers. This reduces computation and avoids overfitting.

First, I loaded the pre-trained MobileNetV2 model:

```
model = models.mobilenet_v2(pretrained=True)
```

Freezing Layers: All layers were frozen first by setting `requires_grad = False`. This stops the gradients from being calculated for those layers:

```
for param in model.parameters():
    param.requires_grad = False
```

Case 1 (FC only): In this case, only the fully connected (FC) layer was trained. I replaced the original FC layer with a new one for 11 classes:

```
num_ftrs = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_ftrs, 11)
```

Then, I made sure the new FC layer is trainable:

```
for param in model.classifier[1].parameters():
    param.requires_grad = True
```

Case 2 (FC and Last 2 Conv Layers): In this case, I trained both the FC layer and the last two convolutional layers:

```
for param in model.features[-1].parameters():
    param.requires_grad = True
for param in model.features[-2].parameters():
    param.requires_grad = True
```

Optimizer and Loss Function: For optimization, I used Adam optimizer this time with different learning rates:

```
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
```

And for loss calculation, I used CrossEntropyLoss:

```
criterion = nn.CrossEntropyLoss()
```

Training: I trained the model for 50 epochs with batch sizes of 32 and 64. Validation accuracy was used to select the best model.

Testing: At the end, I evaluated the model on the test set and drew confusion matrices for each case.

This transfer learning method allowed me to get better performance by fine-tuning only selected parts of the MobileNetV2 model. Also, images were resized to 150x150 pixels like in the previous methods.

Model Code Snippet:

```
# Load pre-trained MobileNetV2
model = models.mobilenet_v2(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final FC layer
num_ftrs = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_ftrs, 11)

# Unfreeze last layers if needed
for param in model.classifier[1].parameters():
    param.requires_grad = True
```

Some Images

Here are some graphs and table representations for understanding the results.

Figure 1 shows the confusion matrix of the first model (CNN). This confusion matrix corresponds to the model that achieved the highest test accuracy. "it is not the highest validation model". The model was trained with a batch size of 64 and a learning rate of 0.005. Last training accuracy was 71.68%, and the validation accuracy was 49.82%. It's Test accuracy is 45.09%.

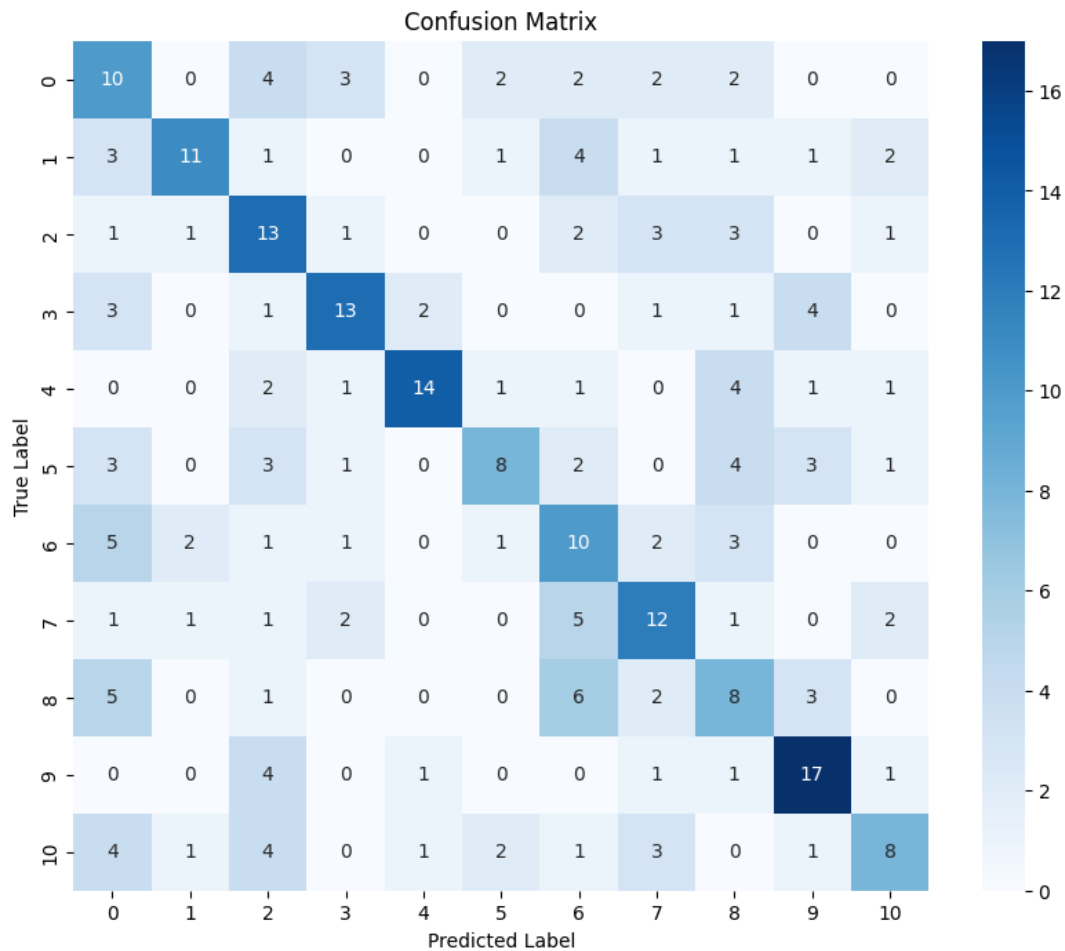


Figure 1: Confusion Matrix of the CNN Model with test accuracy 45.09%

Figure 2 shows the confusion matrix of the CNN model with residuals. This confusion matrix corresponds to the model that achieved the highest test accuracy. "it is not the highest validation model". The model was trained with a batch size of 32 and a learning rate of 0.001. Last training accuracy was 53.23%, and the validation accuracy was 51.27%. It's Test accuracy is 43.64%.

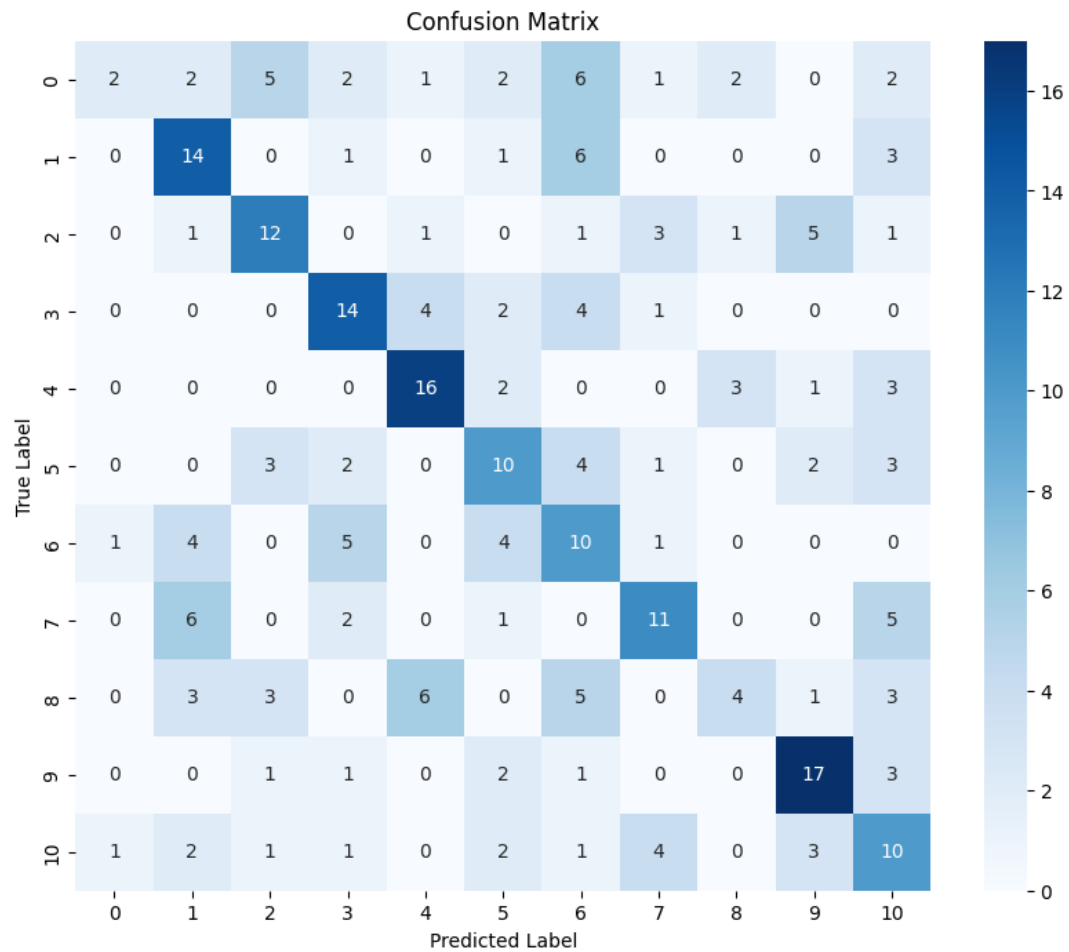


Figure 2: Confusion Matrix of the CNN Model with Residual's Test Accuracy 43.64%

You can see the Loss vs Validation Accuracy graph for the CNN model with batch size of 32 on Figure 3. (Note: These two graphs (Figures 3 and 4) are not included in the code section but the results shown are correct.)

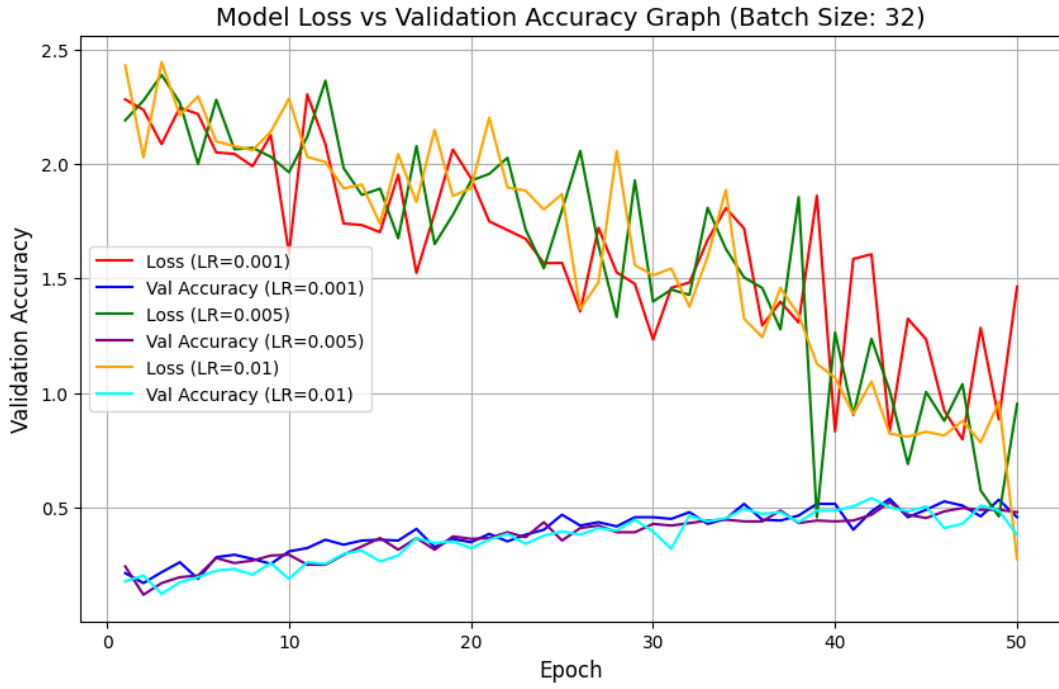


Figure 3: CNN Model's Loss vs Accuracy Graph

You can see the Loss vs Validation Accuracy graph for the CNN model with residuals with batch size of 32 on Figure 4.

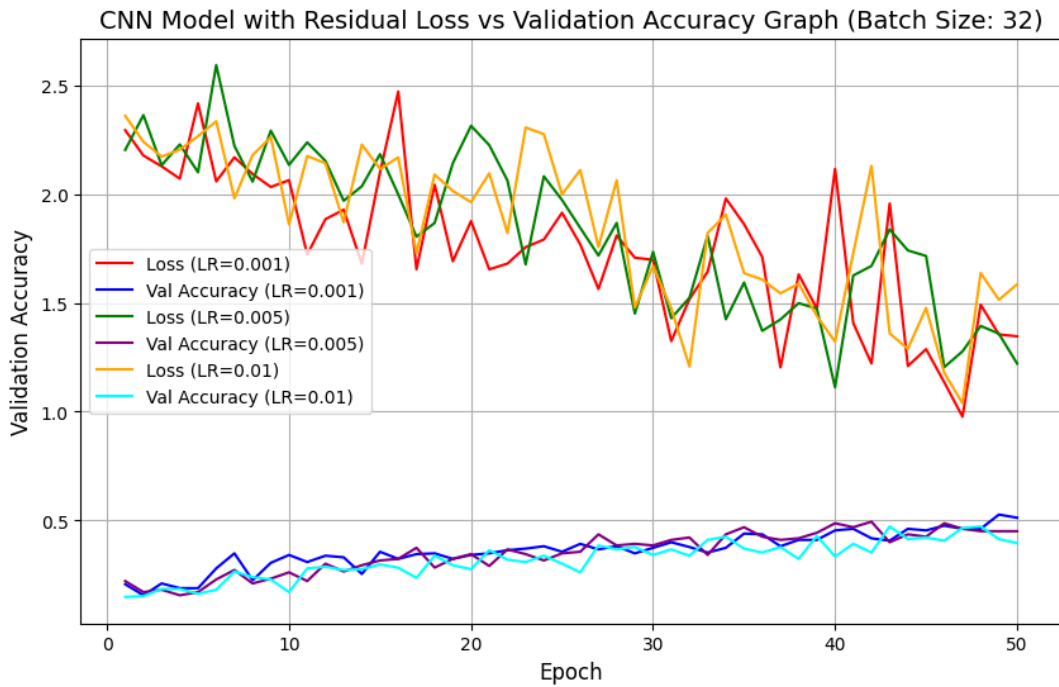
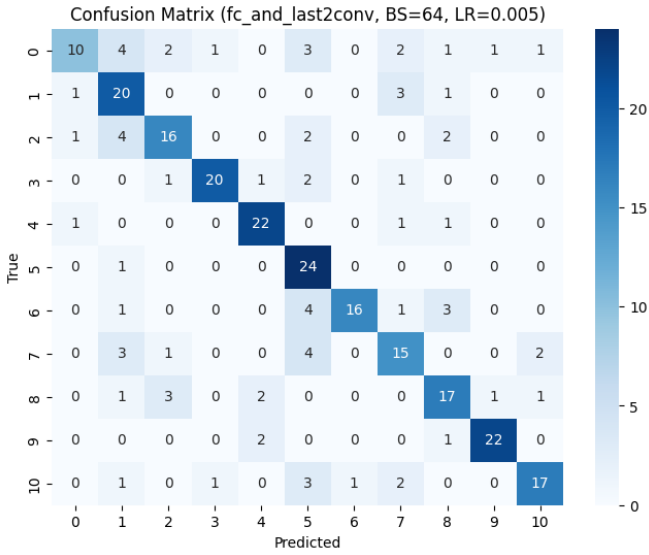
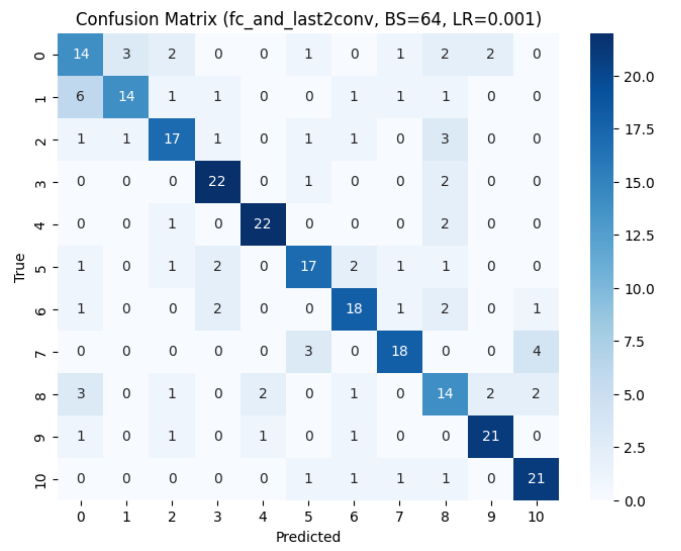


Figure 4: CNN Residual Model's Loss vs Accuracy Graph

In Figure 5, the confusion matrices of the two best models with the highest test accuracies are presented. Model (a) is the transfer learning model where the fully connected layer and the last two convolutional layers are fine-tuned. This model achieved a validation accuracy of 77.82% and a test accuracy of 72.36%. On the other hand, Model (b) uses the same configuration but with a learning rate of 0.001, whereas Model (a) used 0.005. Model (b) achieved a higher validation accuracy of 80.73%.



(a) Test Accuracy 72.36%



(b) Test accuracy 72.00%

Figure 5: The best 2 models of all time. Transfer learning with fully connected and last2 convolutional layer

You can see the Test, Train, and Validation Accuracy graph of the Fine-Tuning model, where only the fully connected (FC) layer was trained with a batch size of 64, in Figure 6. (Note: These two graphs (Figures 6 and 7) are not included in the code section but the results shown are correct.)

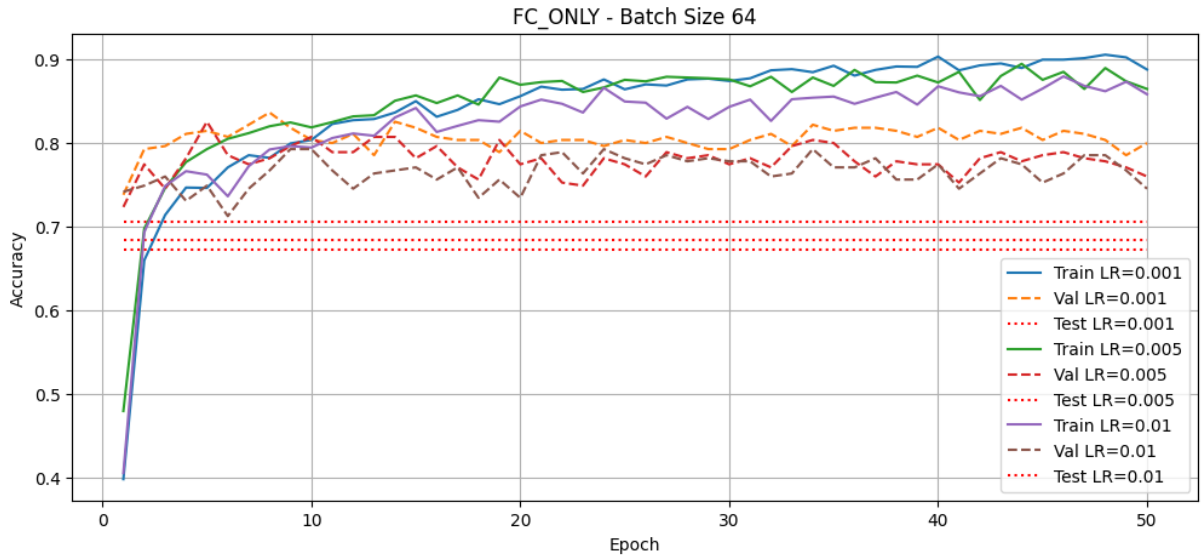


Figure 6: Fine Tuning Model's Test,Train,Validation Accuracy Graph(FC Layer)

Similarly, the Test, Train, and Validation Accuracy graph of the Fine-Tuning model, where both the FC layer and the last two convolutional layers were trained with a batch size of 64, is shown in Figure 7.

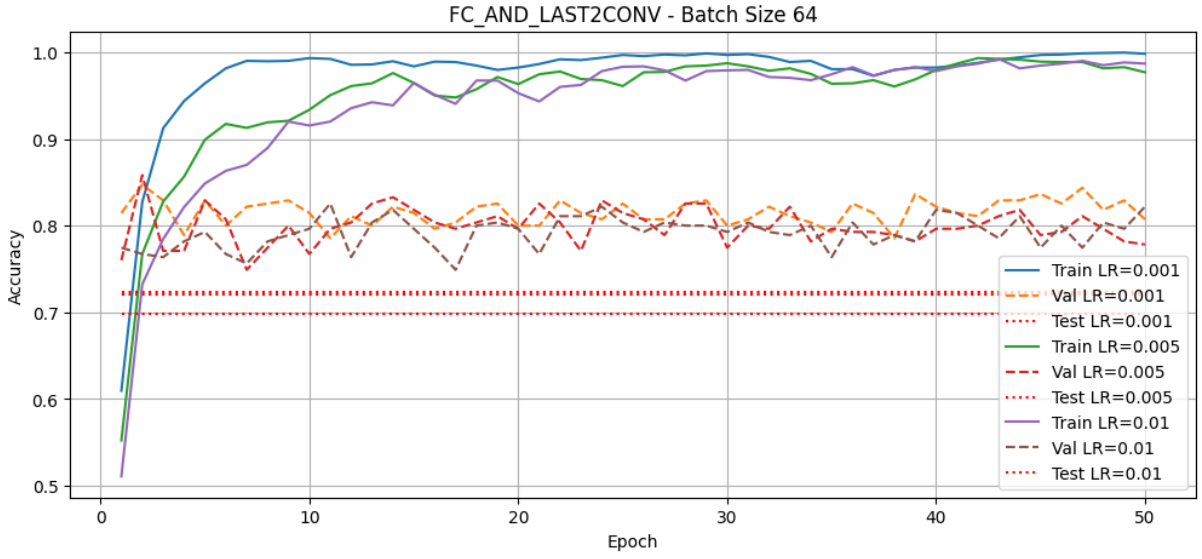


Figure 7: Fine Tuning Model's Test,Train,Validation Accuracy Graph(FC layer and last 2 convolutional layer)

Results

In this section, I present the experimental results obtained from three different model architectures: a baseline Convolutional Neural Network (CNN), a CNN with residual connections, and a transfer learning model using fine-tuning.

- **Baseline CNN:** The simple CNN model trained from scratch achieved a best test accuracy of **45.09%**. While it was able to learn basic features, its performance remained limited, likely due to the relatively small size of the dataset and lack of pretrained knowledge.
- **CNN with Residual Connections:** The addition of residual connections resulted in a best test accuracy of **43.64%**, which is close to the baseline CNN. Although residuals are generally expected to help deeper networks converge better, in this case, the improvement was not significant, and both models produced comparable results.
- **Transfer Learning (Fine-Tuning):** By leveraging a pretrained model and fine-tuning it on the target dataset, a substantial performance boost was observed. Two fine-tuning strategies were applied:
 - *Fine-tuning only fully connected layers (FC)* achieved a best test accuracy of **70.55%**.
 - *Fine-tuning both FC and the last two convolutional layers (FC + Last2Conv)* resulted in the highest test accuracy of **72.36%**.

This clearly outperforms both CNN models trained from scratch, highlighting the effectiveness of transfer learning in scenarios with limited data.

Overall, the experiments show that there is little difference between the baseline CNN and the residual CNN, as their accuracies remained close. However, the transfer learning approach brought a significant improvement, pushing the test accuracy over 70% and demonstrating much better generalization. Fine-tuning both the fully connected and convolutional layers yielded the best result, suggesting that allowing more layers to adapt to the new dataset can further enhance performance.

1. Transfer Learning (72.36%) > CNN (45.09%) $\approx$ Residual CNN (43.64%).
2. Residual connections did not yield a notable performance gain in this particular setup.
3. Fine-tuning both FC and convolutional layers proved to be the most effective strategy, outperforming FC-only fine-tuning (72.36% vs. 70.55%).

You can see all results from experiments on Tables below.

Summary of Results

Case	Batch Size	Learning Rate	Test Accuracy (%)
CNN	32	0.001	40.36
CNN	32	0.005	38.18
CNN	32	0.01	40.36
CNN	64	0.001	35.64
CNN	64	0.005	45.09
CNN	64	0.01	43.64
CNN w Residual	32	0.001	43.64
CNN w Residual	32	0.005	34.18
CNN w Residual	32	0.01	37.09
CNN w Residual	64	0.001	36.73
CNN w Residual	64	0.005	39.27
CNN w Residual	64	0.01	42.91

Table 1: Test Accuracies for Different Fine-Tuning Cases, Batch Sizes, and Learning Rates. Best accuracy is highlighted in green and bold.

Case	Batch Size	Learning Rate	Test Accuracy (%)
FC Only	32	0.001	70.55
FC Only	32	0.005	67.27
FC Only	32	0.01	63.64
FC Only	64	0.001	70.55
FC Only	64	0.005	68.36
FC Only	64	0.01	67.27
FC + Last 2 Conv	32	0.001	71.64
FC + Last 2 Conv	32	0.005	66.91
FC + Last 2 Conv	32	0.01	70.91
FC + Last 2 Conv	64	0.001	72.00
FC + Last 2 Conv	64	0.005	72.36
FC + Last 2 Conv	64	0.01	69.82

Table 2: Test Accuracies for Different Fine-Tuning Cases, Batch Sizes, and Learning Rates. Best accuracy is highlighted in green and bold.

Summary

In summary, this study evaluated multiple deep learning approaches for image classification, including training from scratch with CNNs, incorporating residual connections, and applying transfer learning with fine-tuning. While both the baseline CNN and the residual CNN achieved similar results, the transfer learning strategy significantly improved classification performance. The best accuracy was obtained by fine-tuning both the fully connected and the last two convolutional layers, demonstrating the advantage of adapting pretrained models to the specific target data. Overall, the experiments confirm that transfer learning is highly effective for improving model accuracy, especially when working with limited datasets.